Deep Q Learning Driven Stock Trader Bot

Project Proposal

---

1. Project Overview and Motivation

For this project, we are building a Deep Q Network, or DQN, that will make short term stock trading decisions. Each day, the agent will look at the market and its current portfolio and choose whether to buy, sell, or hold. It will receive a reward based on how the portfolio value changes after trading costs. This is a simulation, so we are not claiming that the model will guarantee real world profits.

We will start with historical SPY ETF data. SPY follows the S&P 500 and can be traded, which makes it a good asset for our simulation. Once the SPY model is working, we may test the same approach on a few additional stocks, such as AAPL, MSFT, AMZN, and TSLA, to see whether it works across different assets.

2. Research Question and Objectives

Our main research question is: Can a DQN agent learn to choose buy, sell, and hold actions that perform competitively against standard trading strategies when tested on new historical data?

- Build a custom trading environment with cash, holdings, portfolio value, and transaction costs.
- Train a DQN using an online Q network, target network, replay buffer, and epsilon greedy exploration.
- Compare DQN with buy and hold, random trading, and a moving average crossover strategy.
- Evaluate return, volatility, maximum drawdown, Sharpe ratio, trade count, and transaction costs.
- Use chronological train/test splitting and repeated random seeds to reduce misleading conclusions.

3. Environment, Data, and RL Method

The agent will observe recent returns, moving average information, trading volume, its current position, available cash, and portfolio value. It will choose one of three actions: buy, sell, or hold. The environment will prevent unrealistic actions, such as buying without enough cash or selling shares that the agent does not own. The reward will be the change in the portfolio after trading costs:

$$r_t = \log(V_{t+1} / V_t) \quad \text{transaction_cost}_t$$

Instead of storing every Q value in a table, the DQN will use a neural network to estimate Q(s,a). The agent will save its previous experiences in a replay buffer and learn from small batches of those experiences. A separate target network will help make training more stable. We will experiment with settings such as the learning rate, discount factor, epsilon schedule, replay buffer size, batch size, target network update frequency, and trading cost.

We will download daily SPY price and volume data from Yahoo! Finance using the yfinance library. We will use earlier data to train the agent and later data to test it. The test period will remain separate and will not be used to choose features or tune the model. We will use the Q Trader repository as a coding reference and cite it in our report.

4. Tools and Background Work

- We will use Python, NumPy, Pandas, Matplotlib, PyTorch, yfinance, Jupyter Notebook, and a custom trading environment based on the Gymnasium style.
- We will use GitHub so that everyone can share code, track changes, and reproduce the results.
- Our background reading will include Sutton and Barto's Reinforcement Learning: An Introduction, the DQN paper by Mnih and colleagues, and the Double Q Learning paper by van Hasselt and colleagues.
- Before running the full experiment, we will check that the data pipeline works, the environment reset and step functions work correctly, and a small DQN training run completes successfully.

5. Project Schedule and Deliverables

Week	Planned work	Expected output

Week 10 Oct 26 Nov 1	Review DQN and trading RL background; clean SPY data; finalize the state, actions, reward, and train/test split.	Project setup and proposal preparation
Week 11 Nov 2 Nov 8	Clean and preprocess the data; Implement the trading environment, replay buffer, DQN network, and target network update.	Initial DQN code
Week 12 Nov 9 Nov 15	Run initial training; debug the environment; implement buy and hold, random, and moving average baselines.	Working experiment pipeline
Week 13 Nov 16 Nov 22	Tune hyperparameters using training data only and produce preliminary results.	Preliminary plots
Week 14 Nov 23 Nov 29	Run final SPY experiments, transaction cost ablations, and repeated seed evaluations; attempt the optional multi stock extension if feasible.	Final experiment tables and figures
Week 15 Nov 30 Dec 6	Analyze results, complete the report, finalize code and the trained model, and prepare the presentation.	Final report and presentation draft
Week 16 Dec 7 Dec 10	Submit the approximately ten page final report, code, trained model, and evaluation form by Dec 8. Present during the scheduled session.	Final submission due Dec 8; presentation

6. Expected Deliverables and Limitations

The **final deliverables** will include reproducible Python code, the trained DQN model, the trading environment, benchmark implementations, experiment results, portfolio value and cumulative return plots, a final report, and a class presentation. **Results** will be interpreted as historical simulation findings only.

Some **limitations** are that markets change over time, the model may overfit the historical data, and our simulation will simplify issues such as slippage and liquidity. Good historical performance also does not guarantee future performance.

References

1. Mnih, V., et al. (2015). Human level control through deep reinforcement learning. Nature, 518, 529 533. https://doi.org/10.1038/nature14236
2. van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q learning. Proceedings of the AAAI Conference on Artificial Intelligence, 30(1). https://doi.org/10.1609/aaai.v30i1.10295
3. Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press. https://mitpress.mit.edu/9780262039246/reinforcement learning/
4. Lu, E. Q Trader: An implementation of Q learning applied to short term stock trading. GitHub. https://github.com/edwardhdlu/q trader
5. Yahoo! Finance. Historical market data. https://finance.yahoo.com/